華南理工大學

South China University of Technology

Computer Network

Lab Report

Name

Student ID

Class

Collaborator(s)

Task 1 Network Traffic Analysis

Purposes and tasks:

（1）Learn the use of wireshark. You can do this task on your own PC and own LAN.

（2）Learn analysis of network traffic caught.

（3）Explain TCP hand-shaking process.

（4）Learn how TCP hand-shaking failure can be handled.

（5）Analysis ARP packets

Steps:

1. Setting up experimental environment (skip this step, if your environment has all been set up)

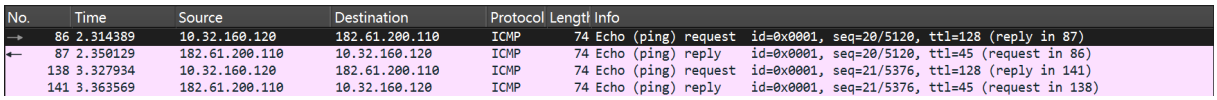
（1）Wireshark

（2）PC connected to network

2. Basic operations on Wireshark

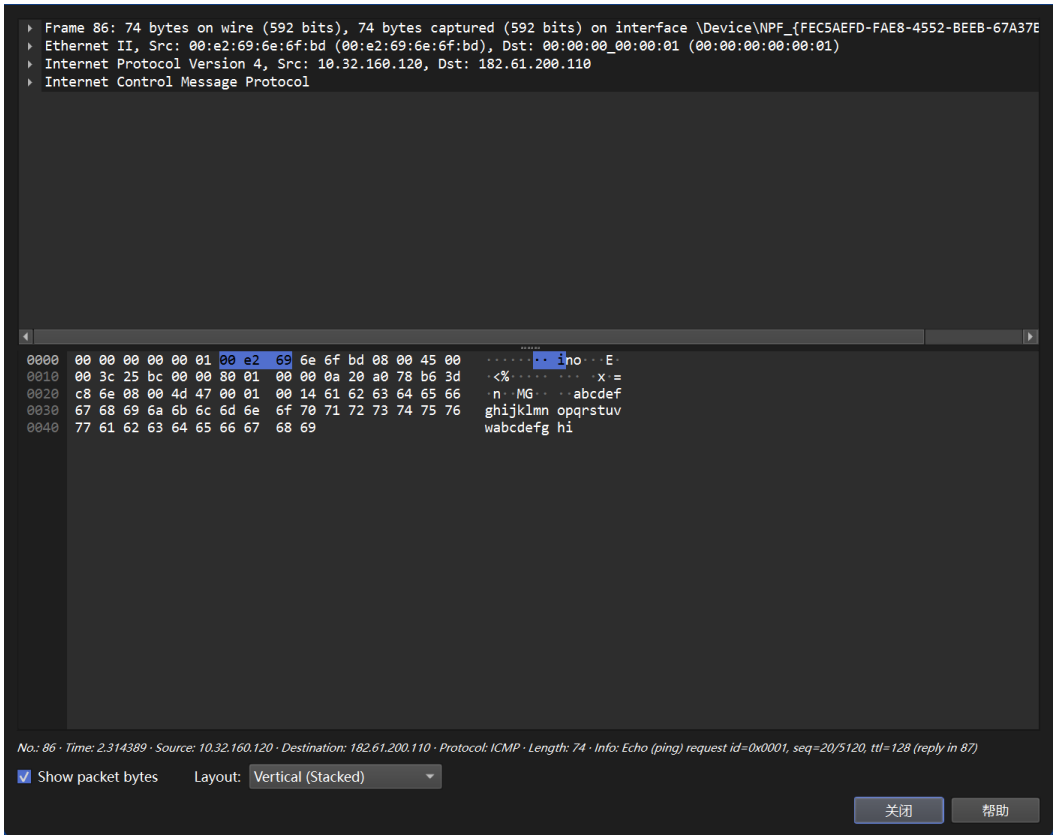
（1）Monitor on a particular network interface

（2）Capture particular packets.



No.	Time	Source	Destination	Protocol	Length	Info
86	2.314389	10.32.160.120	182.61.200.110	ICMP	74	Echo (ping) request id=0x0001, seq=20/5120, ttl=128 (reply in 87)
87	2.350129	182.61.200.110	10.32.160.120	ICMP	74	Echo (ping) reply id=0x0001, seq=20/5120, ttl=45 (request in 86)
138	3.327934	10.32.160.120	182.61.200.110	ICMP	74	Echo (ping) request id=0x0001, seq=21/5376, ttl=128 (reply in 141)
141	3.363569	182.61.200.110	10.32.160.120	ICMP	74	Echo (ping) reply id=0x0001, seq=21/5376, ttl=45 (request in 138)

This is an example for pinging www.baidu.com. The info in the screenshot shows that the packets are from the ping operation. And the following screenshot shows the information of packet No.86.



3. Record an example of a successful TCP three-hand-shaking

188	4.880479	10.32.160.120	123.125.1.102	TCP	66	65224 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
189	4.917106	123.125.1.102	10.32.160.120	TCP	60	80 → 65224 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1424
190	4.917181	10.32.160.120	123.125.1.102	TCP	54	65224 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0

The sequence numbers and acknowledgment numbers increment correctly across the three packets, confirming it's a standard TCP three-way handshake.

4. Record an example of hand-shaking failure

ws.col.protocol == "TCP"

No.	Time	Source	Destination	Protocol	Length	Info
52	1.438063	104.208.16.95	10.32.160.120	TCP	60	443 → 49852 [ACK] Seq=1 Ack=14178 Win=16385 Len=0
55	1.497965	10.32.160.120	104.208.16.95	TCP	54	49852 → 443 [ACK] Seq=14178 Ack=40 Win=254 Len=0
64	1.813093	10.32.160.120	104.208.16.95	TCP	54	49852 → 443 [ACK] Seq=14178 Ack=350 Win=252 Len=0
84	2.770013	13.89.179.10	10.32.160.120	TCP	60	443 → 49881 [ACK] Seq=1 Ack=1126 Win=16385 Len=0
94	2.887545	10.32.160.120	142.250.77.10	TCP	66	50334 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
98	3.045105	13.89.179.10	10.32.160.120	TCP	60	443 → 49881 [ACK] Seq=95 Ack=1246 Win=16384 Len=0
100	3.092048	10.32.160.120	13.89.179.10	TCP	54	49881 → 443 [ACK] Seq=2030 Ack=189 Win=250 Len=0
101	3.154450	10.32.160.120	142.250.77.10	TCP	66	50335 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

This screenshot shows the failure of TCP three-way handshake when we are trying to connect to www.youtube.com. Since the GFW, we cannot access to this website, where the screenshot prove this condition. We can see that in packet No.94, our computer send a packet to the server. And in packet, our computer send a same packet with No.94, which means that the handshake is failed. Because we haven't received any response from the server.

5. Monitoring ARP packets

ws.col.protocol == "ARP"

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.192? Tell 10.32.255.254
5	0.236217	00:00:00_00:00:01	Broadcast	ARP	60	ARP Announcement for 10.36.160.120
6	0.340311	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.128.17? Tell 10.32.255.254
7	0.375714	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.193? Tell 10.32.255.254
10	0.542072	00:00:00_00:00:01	Broadcast	ARP	60	ARP Announcement for 192.168.0.233
12	0.751879	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.194? Tell 10.32.255.254
13	0.761321	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.187? Tell 10.32.255.254
14	0.764978	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.188? Tell 10.32.255.254
15	0.768898	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.189? Tell 10.32.255.254
16	0.772478	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.190? Tell 10.32.255.254
17	0.775992	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.191? Tell 10.32.255.254
19	1.120597	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.195? Tell 10.32.255.254
46	1.244336	00:00:00_00:00:01	Broadcast	ARP	64	Who has 10.32.211.54? Tell 10.32.255.254
49	1.389440	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.128.65? Tell 10.32.255.254
54	1.491724	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.196? Tell 10.32.255.254
56	1.719561	00:00:00_00:00:01	Broadcast	ARP	64	Who has 10.32.138.55? Tell 10.32.255.254
57	1.763482	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.189? Tell 10.32.255.254
59	1.766903	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.190? Tell 10.32.255.254
60	1.771324	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.191? Tell 10.32.255.254
61	1.774799	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.192? Tell 10.32.255.254
62	1.778381	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.193? Tell 10.32.255.254
63	1.781909	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.196.194? Tell 10.32.255.254
67	2.167369	00:00:00_00:00:01	Broadcast	ARP	60	Who has 10.32.128.17? Tell 10.32.255.254

This screenshot shows the computer is sending ARP packets all the time. The destination is "Broadcast", this is also another specification of ARP Protocol.

6. Monitoring ICMP packets

ws.col.protocol == "ICMP"

No.	Time	Source	Destination	Protocol	Length	Info
76395	44.842922	121.224.81.39	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
76476	45.931235	171.115.216.246	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
76491	45.945196	27.189.78.152	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
76492	45.948636	1.84.124.177	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
76735	47.568041	113.79.169.140	10.32.160.120	ICMP	86	Destination unreachable (Host unreachable)
76903	48.939030	121.224.81.39	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
77030	50.578682	113.79.169.140	10.32.160.120	ICMP	86	Destination unreachable (Host unreachable)
77124	51.424268	171.12.11.122	10.32.160.120	ICMP	70	Destination unreachable (Port unreachable)
77293	53.987751	10.32.160.120	182.61.200.108	ICMP	74	Echo (ping) request id=0x0001, seq=70/17920, ttl=128 (reply in 77296)
77296	54.023547	182.61.200.108	10.32.160.120	ICMP	74	Echo (ping) reply id=0x0001, seq=70/17920, ttl=44 (request in 77293)
77338	54.993910	10.32.160.120	182.61.200.108	ICMP	74	Echo (ping) request id=0x0001, seq=71/18176, ttl=128 (reply in 77339)
77339	55.029700	182.61.200.108	10.32.160.120	ICMP	74	Echo (ping) reply id=0x0001, seq=71/18176, ttl=44 (request in 77338)
77427	56.002201	10.32.160.120	182.61.200.108	ICMP	74	Echo (ping) request id=0x0001, seq=72/18432, ttl=128 (reply in 77433)
77433	56.037923	182.61.200.108	10.32.160.120	ICMP	74	Echo (ping) reply id=0x0001, seq=72/18432, ttl=44 (request in 77427)
77505	56.628693	113.79.169.140	10.32.160.120	ICMP	86	Destination unreachable (Host unreachable)
77516	56.645054	27.187.14.145	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
77522	56.654039	14.145.169.181	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
77540	56.659084	180.114.147.198	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
77541	56.660736	113.141.119.11	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
77548	56.672591	39.168.152.181	10.32.160.120	ICMP	86	Destination unreachable (Port unreachable)
77590	57.016118	10.32.160.120	182.61.200.108	ICMP	74	Echo (ping) request id=0x0001, seq=73/18688, ttl=128 (reply in 77592)
77592	57.051862	182.61.200.108	10.32.160.120	ICMP	74	Echo (ping) reply id=0x0001, seq=73/18688, ttl=44 (request in 77590)

This screenshot shows the ICMP packets between our computer and www.baidu.com. The “P rotocol” in the pictures proves that these packets are ICMP packets.

Task 2 Basic Router Operations & static Routing

1. Experimental Purposes

（1）Get familiar with Cisco PacketTracer and router

（2）Learn basic router operation command

（3）Static routing

2. Please answer following questions in your lab report

（1）What is a router

A router is a networking device that forwards data packets between computer networks.

（2）How can we remotely log onto a router? What tools can be used? How could you configure these tools?

We can remotely log onto a router using SSH or Telnet. Tools like PuTTY can be used, configured by entering the router's IP address and selecting the connection type.

（3）What are the five working modes of a router? Prompts? Meaning/function? How can you switch between different modes?

Router working modes include User mode, Privilege mode, Global configuration mode, Interface configuration mode, Router configuration mode. We use “enable” to enter the Privilege mode, “configure terminal” to enter the Global mode, “interface [port]” to enter the Interface mode, “router [protocol” to enter the Router mode. We can use “exit” to return to the previous mode, and use “end” to back to the Privilege mode.

User mode allows basic information viewing, Privilege mode offers advanced viewing and diagnostic capabilities, Global configuration mode is for router-wide general settings, Interface configuration mode focuses on configuring individual network ports, and Router configuration mode is dedicated to setting up and managing routing protocols.

（4）Describe the steps/commands of setting IP address on a router interface

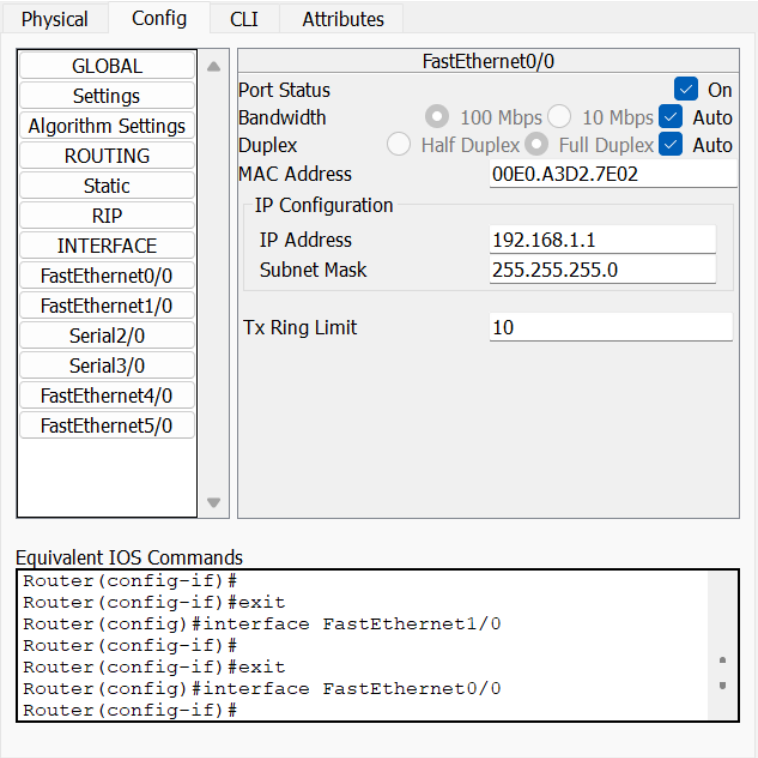
We need to use “int fa 0/1” to enter the FastEthernet 1/0 port, then enter “ip address 192.168.114.1 255.255.255.0” to config the ipv4 address and the subnet mask. (The address and mask are just for example.)

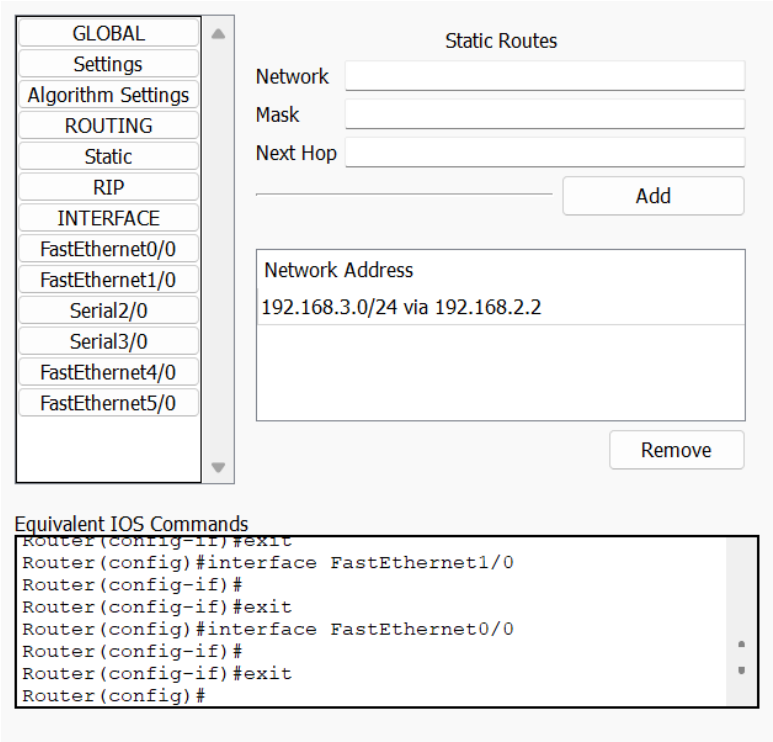
（5）Explain how ‘show’ command can be applied to obtain router information interface information/configuration

“show” is an important command designed for display various operational states and configuration details. The following screenshots shows how to use “show” command to display information like routes and interface status.

```
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M -
mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA -
OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA
external type 2
       E1 - OSPF external type 1, E2 - OSPF external
type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS
level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route,
o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

C    192.168.1.0/24 is directly connected,
FastEthernet0/0
C    192.168.2.0/24 is directly connected, Serial2/0
S    192.168.3.0/24 [1/0] via 192.168.2.2

Router#show ip interface brief
Interface              IP-Address      OK? Method Status
Protocol
FastEthernet0/0        192.168.1.1     YES manual up
up
FastEthernet1/0        unassigned      YES unset
administratively down down
Serial2/0              192.168.2.1     YES manual up
up
Serial3/0              unassigned      YES unset
administratively down down
FastEthernet4/0        unassigned      YES unset
administratively down down
FastEthernet5/0        unassigned      YES unset
administratively down down
```

（6）Describe how routing entries can be manually configured.

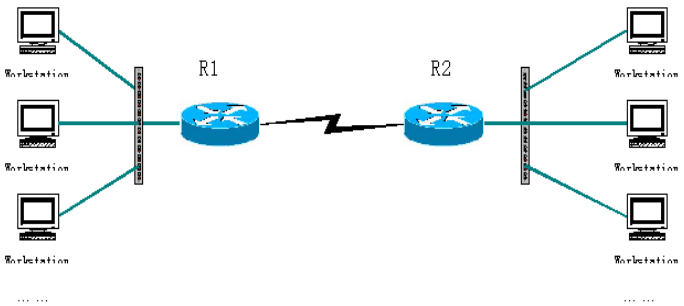




The screenshots show how to configure the routing entries manually. First we need to enter the graphical interface and enter the ipv4 address and subnet mask to finish the IP configuration. Select "Static" on the left side of the interface, and now we have to configure the static routes by entering address, mask, and next hop.

2. Experimental Environment

（1）Network topology (for example)



（2）Experiment devices

Two routers, two switches (optional), two PCs and cables.

（3）Experiment requirements

Given 3 C-class Network addresses: 192.168.1.0，192.168.2.0，192.168.3.0. (<u>For example only. You can design the networks as you like</u>.)

A. Using the above topology to design network, explain the address for each node and interface. Please describe the commands/steps to configure these IP addresses.

The command to configure ipv4 address is "int fa 0/1" and "ip address 192.168.114.1 255.255.255.0"(for PC 0 in the following picture).

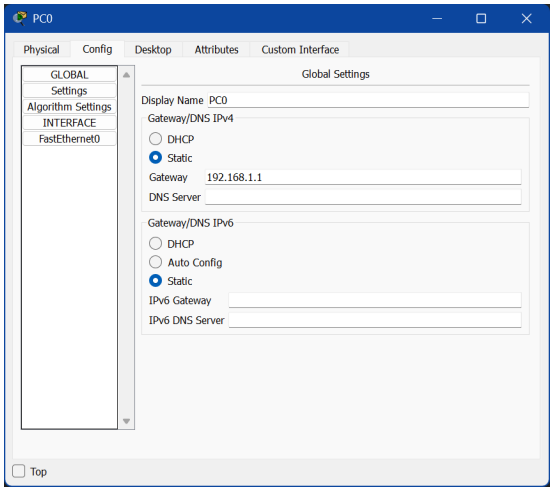
B. Using static routing, to enable the workstations to communicate with each other. Explain the routing entries on each router. Explain the commands/steps to configure routing entries.

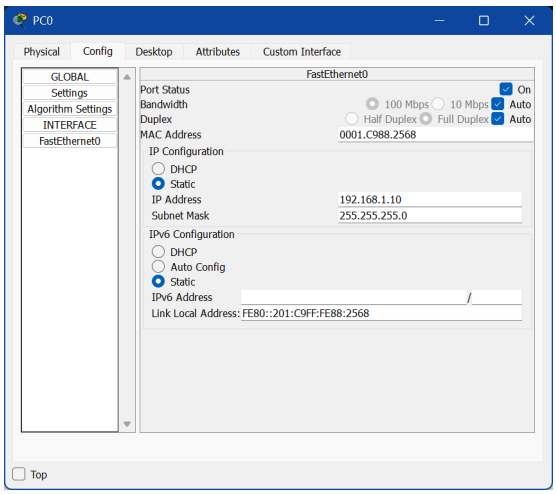
We use “ip route 192.168.3.0 255.255.255.0 192.168.2.2” for Router 2 and “ip route 192.168.1.0 255.255.255.0 192.168.2.1” for Router 3 to configure the static routing.

We use two routers and two computers to finish this lab. The topology is in the following screenshot.

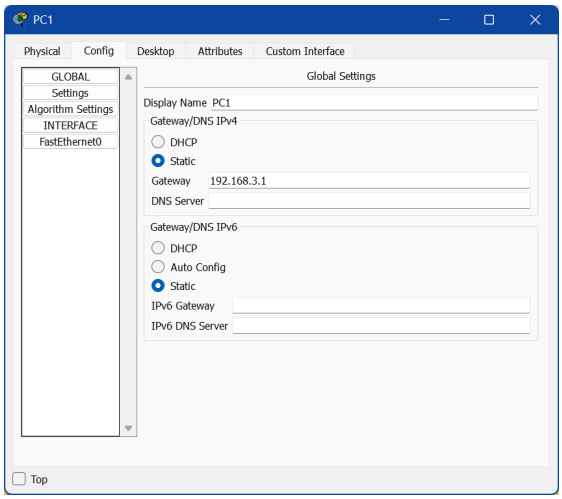
The configurations are in these pictures.

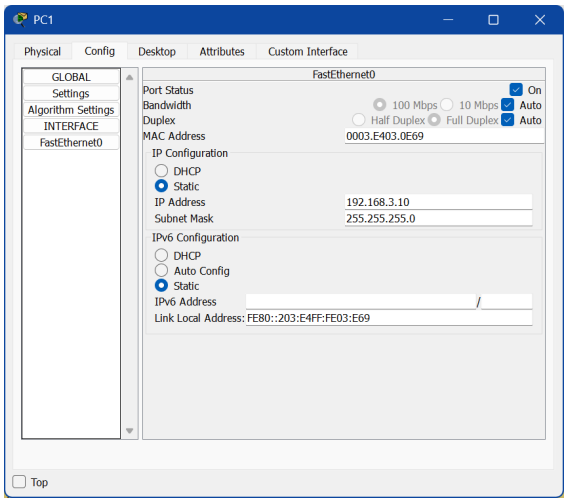
PC0:



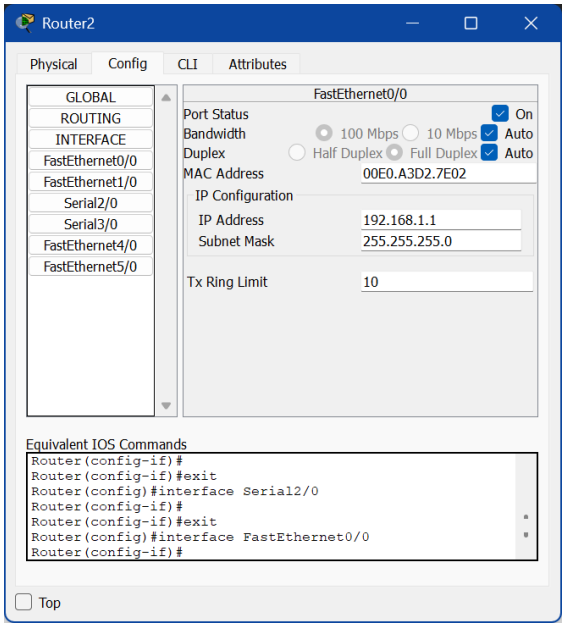


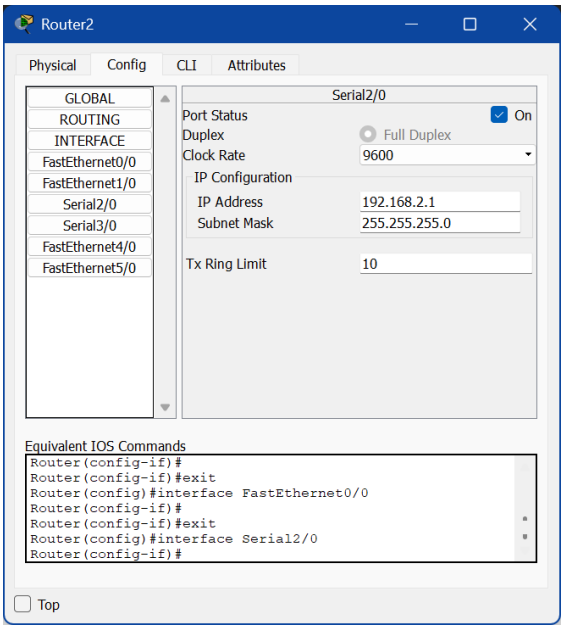
PC1:



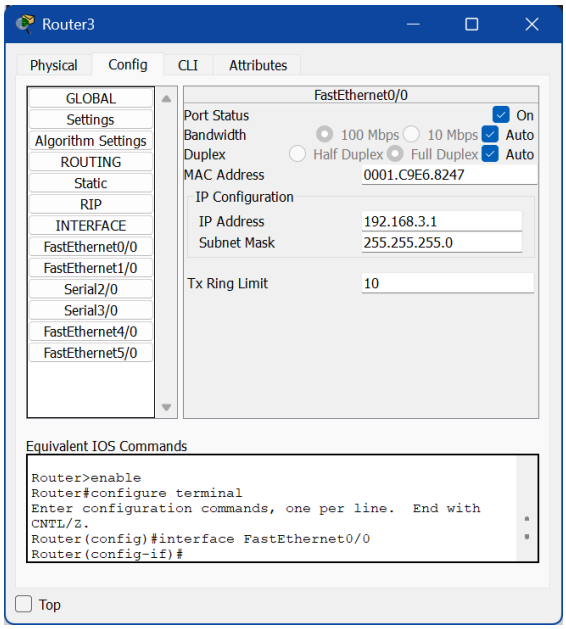


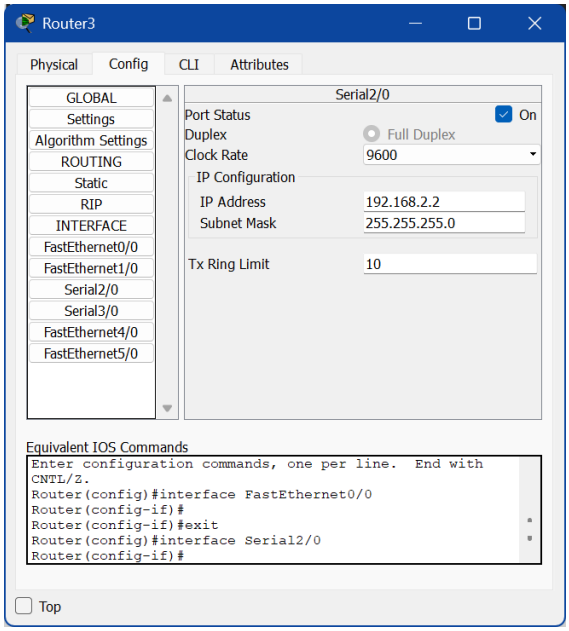
Router 2:



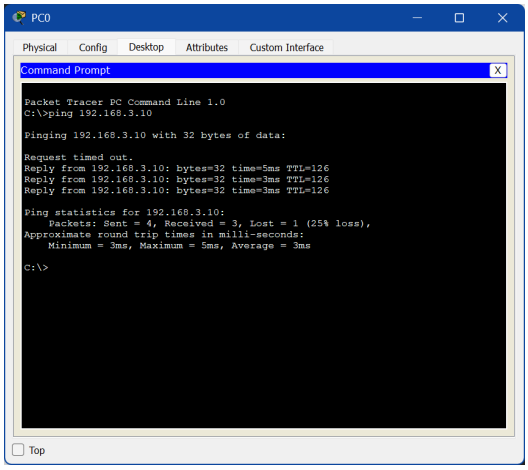


Router 3:





Try to ping PC1 from PC0, success.



Task 3 Networking --- Dynamic routing

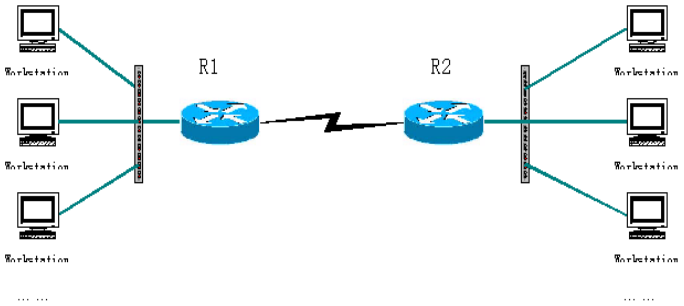
1. Experimental Purposes

（1）RIP configuration

（2）OSPF configuration

2. Experimental Environment

（1）Network topology (for example)



（2）Experiment devices

Two routers, two switches (optional), two PCs and cables.

（3）Experiment requirements

Given 3 C-class Network addresses: 192.168.1.0，192.168.2.0，192.168.3.0. (For example only. You can design the networks as you like.)

A. Using the above topology to design network, explain the address for each node and interface.

The basic configuration is the same as lab2, including the topology, ipv4 address, subnet mask, port, etc. You can click here to have a look.

B. Using RIP, to enable the workstations to communicate with each other. Explain RIP. Explain the commands/steps to configure dynamic routing.

RIP (Routing Information Protocol) is a distance-vector routing protocol suitable for small networks. It measures path quality based on hop count, with a maximum of 15 hops, beyond which a destination is considered unreachable. RIP periodically broadcasts its entire routing table to adjacent routers.

We need to delete the static routing created in lab2. Then enter “version 2” to configure the RIP version first. For Router 2, we use “network 192.168.1.0” and “network 192.168.2.0”. And for Router 3, we use “network 192.168.2.0” and “network 192.168.3.0”.

C. Using OSPF routing, to enable the workstations to communicate with each other.

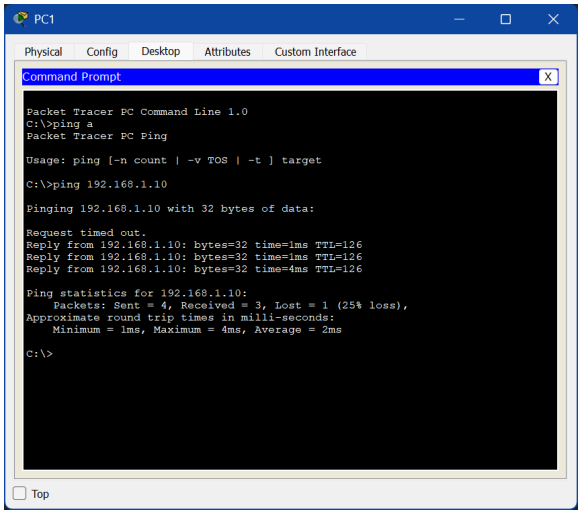
Explain OSPF. Explain the commands/steps to configure dynamic routing.

OSPF (Open Shortest Path First) is a link-state routing protocol designed for large enterprise networks. It calculates the shortest path by building a complete network topology map (Link State Database), offers fast convergence, and supports Classless Inter-Domain Routing (CIDR) and VLSM.

We need to delete the RIP routing first, and then use “router ospf 1” to starts the OSPF routing process with process ID 1 for both two Routers. For Router 2, we use “network 192.168.1.0 0.0.0.255 area 0” and “network 192.168.2.0 0.0.0.255 area 0” to advertises 192.168.1.0/24 and 192.168.2.0/24 into OSPF in area 0. And for Router 3, we use “network 192.168.3.0 0.0.0.255 area 0” and “network 192.168.2.0 0.0.0.255 area 0” to advertises 192.168.3.0/24 and 192.168.2.0/24 into OSPF in area 0.

（4）Testing and confirmation

Try to ping PC0 from PC1, success.



Task 4 Socket Programming

1. Project Purpose

(1) Master the basic principles and methods of Socket network programming

(2) Understand the underlying principle of operation of Socket deeply

2. Project Content

2.1 Requirement

1) Basic: Implement a client and a server similar with ftp protocol; complete the basic function of file transfer.

List the files/directories

Change working-directory

Download file/s

Upload files/s

Create/remove files/directories

Etc…

2) Live demo

High-layer encapsulation Socket can not be used (like java class library, MFC and so on)

2.2 Reference Materials

Recommended reference books:

《UNIX Network Programming》Richard Stevens

《UNIX Environment Advanced Programming》

《TCP/IP Networking technology Volume III（Winsock edition）》

3. Project Environment

（1）Platform: Linux or Windows

（2）Developing Language: C or C++

（3）Developing Environment: no limitation, Visual C++, .NET, Vi, etc.

4. Explanation requirements

Explain the design of your file transfer program.

Explain the usage of socket.h or winsock.h

1. Environment

System: Windows 11 24H2 26100.3775

Language: C++20

IDE: Visual Studio 2022

2. Result

The display of Host:

```
PS E:\Code\lab4\x64\Debug> ./lab4.exe
==================================================
     Network File Hub System (v1.0)
==================================================
  1. Launch Host Mode (Storage: host_storage)
  2. Launch Client Mode (Storage: client_storage)
Please select an option (1 or 2): 1
--- File Nexus Host ---
Enter listening port (default: 9090): Enable logging to file? (y/n, default: n): y
File logging is enabled. Output will be saved to nexus_host_9090.log
[SYSTEM] Created directory: host_storage
2025-06-30 21:17:34.223 [SUCCESS] [SYSTEM] File Nexus Host is now online, listening on port 9090
2025-06-30 21:17:34.226 [INFO] [SYSTEM] Storage root is located at: host_storage

--- Host Activity Log ---
2025-06-30 21:17:44.708 [SUCCESS] [127.0.0.1:62745] Connection established.
2025-06-30 21:17:44.709 [INFO] [127.0.0.1:62745] Client session started. Root: host_storage
2025-06-30 21:17:44.709 [INFO] [127.0.0.1:62745] Request: GET_PATH
2025-06-30 21:17:44.710 [INFO] [127.0.0.1:62745] PWD request: /.
2025-06-30 21:18:15.903 [INFO] [127.0.0.1:62745] Request: LIST_FILES
2025-06-30 21:18:15.904 [INFO] [127.0.0.1:62745] LIST on /. (0 dirs, 0 files)
2025-06-30 21:18:20.417 [INFO] [127.0.0.1:62745] Request: CREATE_DIR, Args: c
2025-06-30 21:18:20.417 [SUCCESS] [127.0.0.1:62745] MKDIR complete: Created c
2025-06-30 21:18:22.659 [INFO] [127.0.0.1:62745] Request: CHANGE_DIR, Args: c
2025-06-30 21:18:22.660 [INFO] [127.0.0.1:62745] CHDIR successful. New path: /c
2025-06-30 21:18:22.661 [INFO] [127.0.0.1:62745] Request: GET_PATH
2025-06-30 21:18:22.661 [INFO] [127.0.0.1:62745] PWD request: /c
2025-06-30 21:18:57.517 [INFO] [127.0.0.1:62745] Request: UPLOAD, Args: test
2025-06-30 21:18:57.518 [INFO] [127.0.0.1:62745] UPLOAD initiated for test
2025-06-30 21:18:57.519 [INFO] [127.0.0.1:62745] Receiving test (0 bytes).
2025-06-30 21:18:57.519 [SUCCESS] [127.0.0.1:62745] UPLOAD complete for test
2025-06-30 21:19:02.970 [INFO] [127.0.0.1:62745] Request: LIST_FILES
2025-06-30 21:19:02.971 [INFO] [127.0.0.1:62745] LIST on /c (0 dirs, 1 files)
2025-06-30 21:19:37.913 [INFO] [127.0.0.1:62745] Request: DELETE_FILE, Args: test
2025-06-30 21:19:37.913 [SUCCESS] [127.0.0.1:62745] DELETE complete: Removed test
2025-06-30 21:19:39.642 [INFO] [127.0.0.1:62745] Request: LIST_FILES
2025-06-30 21:19:39.642 [INFO] [127.0.0.1:62745] LIST on /c (0 dirs, 0 files)
2025-06-30 21:19:43.346 [INFO] [127.0.0.1:62745] Request: CHANGE_DIR, Args: ../
2025-06-30 21:19:43.347 [INFO] [127.0.0.1:62745] CHDIR successful. New path: /
2025-06-30 21:19:43.348 [INFO] [127.0.0.1:62745] Request: GET_PATH
2025-06-30 21:19:43.348 [INFO] [127.0.0.1:62745] PWD request: /
2025-06-30 21:19:44.724 [INFO] [127.0.0.1:62745] Request: LIST_FILES
2025-06-30 21:19:44.724 [INFO] [127.0.0.1:62745] LIST on / (1 dirs, 0 files)
2025-06-30 21:19:50.470 [INFO] [127.0.0.1:62745] Request: REMOVE_DIR, Args: c
2025-06-30 21:19:50.470 [SUCCESS] [127.0.0.1:62745] RMDIR complete: Removed c
2025-06-30 21:19:52.381 [INFO] [127.0.0.1:62745] Request: LIST_FILES
2025-06-30 21:19:52.381 [INFO] [127.0.0.1:62745] LIST on / (0 dirs, 0 files)
```

The display of the User:

```
PS E:\Code\lab4\x64\Debug> ./lab4.exe
==============================================
     Network File Hub System (v1.0)
==============================================
  1. Launch Host Mode (Storage: host_storage)
  2. Launch Client Mode (Storage: client_storage)
Please select an option (1 or 2): 2
--- File Nexus Client ---
Enter host IP address (default: 127.0.0.1): Enter host port (default: 9090): 9090
[SYSTEM] Created directory: client_storage
Successfully connected to host at 127.0.0.1:9090
Your local files are in: 'client_storage'

--- Welcome to the File Nexus Client ---
Available commands: dir, enter <dir>, download <file>, upload <file>, newdir <dir>, deldir <dir>, erase <file>, path, exit

nexus-client [/.]> ls -a
Host Directory Listing [/.]:


nexus-client [/.]> mkdir c
Host Response: Directory created.

nexus-client [/.]> cd c
Directory changed to /c

nexus-client [/c]> upload test
Host is ready. Uploading 'test' from 'client_storage'...
Host Response: File uploaded successfully.

nexus-client [/c]> ls -a
Host Directory Listing [/c]:
       test


nexus-client [/c]> erase test
Host Response: File deleted.

nexus-client [/c]> ls -a
Host Directory Listing [/c]:


nexus-client [/c]> cd ../
Directory changed to /

nexus-client [/]> ls
Host Directory Listing [/]:
<DIR>  c


nexus-client [/]> deldir c
Host Response: Directory removed.

nexus-client [/]> ls -a
Host Directory Listing [/]:


nexus-client [/]>
```

In this screenshot, we test some basic command of our system, like “mkdir”, “upload”, “erase”, “deldir”. And the output shows it works good.

3. Architecture

Main program

```
│
├──── FileNexusHost
│     ├──── BeginAccepting()
│     ├──── ClientSessionHandler()
│     └──── ProcessXXXRequest()
│
├──── FileNexusClient
      ├──── EstablishConnection()
      ├──── StartInteractiveSession()
      └──── IssueXXXCommand()
```

4. Code

1. About the program: The file transfer program is designed based on a classic Client-Server architecture using the TCP protocol for reliable, connection-oriented communication.

2. About winsock: In this program, winsock2.h is the core library that provides the API for all network communication on the Windows platform. It's the Windows equivalent of the POSIX socket.h.

Here are the kernel parts of the program(just the structure):

1. main() Function

```
int main() {
    int choice = 0;
    std::cin >> choice;
    std::cin.ignore();

    switch (choice) {
    case 1:
        LaunchHostMode();
        break;
    case 2:
        LaunchClientMode();
        break;
    default:
        std::cout << "Invalid choice." << std::endl;
        break;
    }
    return 0;
}
```

2. FileNexusHost Class Functions

```
bool Prepare(int port);
void BeginAccepting(int port);
void CeaseOperations();
void ClientSessionHandler(SOCKET clientSocket, const std::string clientAddr);

void ProcessCreateDirRequest(...);
void ProcessRemoveDirRequest(...);
void ProcessDeleteFileRequest(...);
void ProcessChangeDirRequest(...);
void ProcessListRequest(...);
void ProcessDownloadRequest(...);
void ProcessUploadRequest(...);
void ProcessGetPathRequest(...);
```

3. FileNexusCliend Class Functions

```
bool EstablishConnection(const std::string& hostIP, int port);
void StartInteractiveSession();
void EndConnection();
void RefreshRemotePath();
void PrintPrompt();

void IssueListCommand();
void IssueChangeDirCommand(const std::string& dir);
void IssueDownloadCommand(const std::string& filename);
void IssueUploadCommand(const std::string& filename);
void IssueGenericCommand(OperationCode op, const std::string& arg);
void IssueQuitCommand();
```

4. NetIOHandler Class

```
class NetIOHandler {
public:
    static bool TransmitPacket(SOCKET socket, const TransferPacket& packet);
    static bool ReceivePacket(SOCKET socket, TransferPacket& packet);
    static bool StreamFileToSocket(SOCKET socket, const std::string& filepath);
    static bool StreamFileFromSocket(SOCKET socket, const std::string& savePath, long fileSize);
};
```

5. PathManager Class

```
class PathManager {
public:
    static bool IsPathWithinBounds(const std::string& base, const std::string& target);
    static std::string GetRelativeLocation(const std::string& base, const std::string& current);
    static bool EnsureDirectoryExists(const std::string& dirPath);
};
```

6. EventLogger Class

```
class EventLogger {
public:
    enum class Level { Info, Warn, Error, Success, System };

    static void Info(const std::string& client, const std::string& msg);
    static void Error(const std::string& client, const std::string& msg);
    static void Success(const std::string& client, const std::string& msg);
    static void Warn(const std::string& client, const std::string& msg);
    static void System(Level level, const std::string& msg);
    static void EnableFileLogging(bool enable, const std::string& filename);
};
```